

How To Use This Document AI Agent Guide

★ This document is a dual-purpose artefact. It serves as a comprehensive implementation plan for the DawaaLink project AND as a structured prompt feed for an AI coding agent. Every section contains both human-readable specifications AND agent-ready directive boxes (purple background) that can be copy-pasted directly into an AI agent session. Follow milestones in order. Each milestone is self-contained and produces verifiable deliverables before the next begins.

Document Structure:

Section	Content	Page Role
0	System Context & Architecture Brief	Orientation — read first
1	Technology Stack Decisions	Configuration
2	Database Schema & ERD	Agent: generate migration files
3	Milestone 1 — Auth & Registration	Agent: implement + test
4	Milestone 2 — Dead Stock Listing	Agent: implement + test
5	Milestone 3 — Matching Engine	Agent: implement + test
6	Milestone 4 — Swap Workflow	Agent: implement + test
7	Milestone 5 — Swap Records & Audit	Agent: implement + test
8	Milestone 6 — Dashboard & Alerts	Agent: implement + test
9	Milestone 7 — Frontend (React SPA)	Agent: implement
10	Milestone 8 — Integration, I18n & NFRs	Agent: harden
11	API Contract Reference	Always-on reference
12	Controlled Substance Exclusion List	Hard constraint
13	Acceptance Criteria Master Checklist	QA gate
14	Risk Register & Mitigations	Ongoing

Section 0 — System Context & Architecture Brief

Read First

What DawaaLink Is

DawaaLink is a web-based pharmacy inventory exchange platform that enables independent pharmacies in Greater Cairo to exchange near-expiry dead stock with each other through an automated, cash-free, multi-party barter system. It formalises an existing informal behaviour (WhatsApp-based stock swaps, confirmed in 9/14 surveyed pharmacies) into a governed digital platform.

Core Problem Statement

! Pharmacies consistently lose revenue to expired medications that could not be sold in their current location but had demand elsewhere. DawaaLink matches pharmacies holding excess near-expiry stock of medicine X with pharmacies that need medicine X — automatically, within a geographic radius, with full documentation, and zero cash changing hands.

Architecture Overview

Layer	Technology	Status	Notes
Frontend	React (SPA)	CONFIRMED	Single Page App, desktop-first, mobile-responsive
REST API	Java 17 / Spring Boot	CONFIRMED	RESTful, JWT auth, OpenAPI 3.0 docs
Real-time	WebSocket (/ws/notifications)	CONFIRMED	Swap ticket delivery, live dashboard
Database	PostgreSQL	CONFIRMED	9 entities, Flyway migrations
Matching Engine	Java service — DFS / Johnson's algorithm	CONFIRMED	Separate service component, graph cycle detection
Geolocation	OpenStreetMap / Nominatim	CONFIRMED	Free, open-source, distance calc
Email/Notif	SMTP (Resend or similar)	TBD — sprint 1	Swap alerts, 24h reminders
PDF Generator	iText / Apache PDFBox / JasperReports	TBD	Swap Record PDF generation
Drug Reference DB	Team-provided controlled substances dataset	TBD — sprint 2	Flagging controlled substances
Hosting	Cloud (AWS / GCP / Azure)	TBD	Managed DB, 99% uptime SLA
Schema Migrations	Flyway or Liquibase	To be confirmed	Required per NFR

Hard Constraints — Non-Negotiable

- **CONTROLLED SUBSTANCES:** Categorical exclusion enforced at DB level. No morphine, codeine, benzodiazepines, or any EDA Schedule I/II/III drug may be listed. See Section 12.
- **CASH-FREE:** No payment processing of any kind. Barter only.
- **PRIVACY:** Inventory data visible only to matched swap partners. Never public.
- **GEOGRAPHY:** Greater Cairo only for v1. No cross-governorate matching.
- **BACKEND:** Java / Spring Boot. Non-negotiable per academic constraint.
- **COLD-CHAIN:** Refrigerated drugs excluded. Storage condition unverifiable remotely.
- **DEADLINE:** June 2026. No scope extensions assumed.

Section 1 — Technology Stack & Open Decisions

Configuration

AI AGENT DIRECTIVE – Section 1: Before writing any code, confirm the two open decisions below with the team (PDF library and notification provider). Once confirmed, update the Spring Boot pom.xml dependencies accordingly and create a project scaffold with the following structure: backend/ – Spring Boot (Maven) frontend/ – React (Vite) db/ – Flyway migration scripts docs/ – OpenAPI spec Use Java 17, Spring Boot 3.x, Spring Security (JWT), Spring Data JPA, and WebSocket (STOMP over SockJS).

Open Decisions Requiring Team Sign-Off:

Decision	Options	Recommendation	Must Decide By
PDF Generator	iText 7, Apache PDFBox, JasperReports	Reportiz (most feature-rich, good for tables)	Sprint 1, Week 1
Notification Service	Resend (email), SendGrid, AWS SES, Twilio SMS	Resend, the simplest API, free tier sufficient	Sprint 1, Week 1
Drug Reference DB	Team CSV, WHO GTIN dataset, EDA with hardcoded EDA Schedule list (Section 12), expand in sprint 2	Start with hardcoded EDA Schedule list (Section 12), expand in sprint 2	Sprint 2
Cloud Hosting	AWS (EC2+RDS), GCP, Azure, Render.com	Render.com for MVP (simplest deployment)	Before final demo

Confirmed Spring Boot Dependencies (pom.xml core):

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-security
spring-boot-starter-websocket
spring-boot-starter-validation
spring-boot-starter-mail
io.jsonwebtoken:jjwt-api:0.12.x
org.postgresql:postgresql
org.flywaydb:flyway-core
org.springdoc:springdoc-openapi-starter-webmvc-ui:2.x
[iText 7 or PDFBox – TBD]
```

Section 2 — Database Schema (9 Entities) PostgreSQL +

Flyway

AI AGENT DIRECTIVE – Section 2: Generate Flyway migration script V1__init_schema.sql implementing ALL 9 entities below exactly as specified. Use UUID primary keys (gen_random_uuid()). Add indexes on: pharmacy_id, gtin, expiry_date, lock_status, execution_status. Enforce all ENUM types via PostgreSQL CHECK constraints. Implement the AuditLog table with NO DELETE and NO UPDATE privileges (row-level security). After the schema, generate JPA @Entity classes for all 9 entities with proper @ManyToOne, @OneToMany relationships.

e, address, tax_id, status ENUM(PENDING/ACTIVE/SUSPENDED), latitude, longitude
assword_hash, role ENUM(OWNER/PHARMACIST/EMPLOYEE), created_at
syndicate_number, eda_registration, verification_status
manufacturer, storage_condition ENUM(ROOM_TEMP/COLD_CHAIN), is_controlled BOOLEAN
n_number, expiry_date (DATE), quantity_available (INT), unit ENUM(BOX/STRIP/VIAL/PIECE), unit_price (DECIMAL), lock_status ENUM(ACTIVE/LOCKED/FLAGGED)
y_needed (INT), min_acceptable_expiry (DATE)
ion_status ENUM(PENDING/PARTIAL_ACCEPT/CONFIRMED/IN_TRANSFER/COMPLETED/CANCELLED/TIMED_OUT), expires_at (TIMESTAMP), hash_signature
, receiver_pharmacy_id (FK), inventory_id (FK), quantity_transferred (INT), leg_status ENUM(PENDING/ACCEPTED/DECLINED/COMPLETED)
AR), target_entity_id (UUID), timestamp, ip_address — WRITE ONLY, no UPDATE/DELETE

Critical Database Rules:

- InventoryItem.lock_status transitions: ACTIVE → LOCKED (on match) → ACTIVE (on decline/timeout) or COMPLETED (on swap done). FLAGGED is terminal until admin review.
- SwapCycle.expires_at = created_at + 72 hours. Scheduled job checks every 15 minutes.
- AuditLog: Grant INSERT only on this table. REVOKE UPDATE, DELETE from all roles.
- MedicationReference.is_controlled = TRUE automatically blocks listing (status = FLAGGED).
- storage_condition = COLD_CHAIN on InventoryItem triggers automatic exclusion from matching pool.
- Unique constraint on Pharmacy.tax_id to prevent duplicate registrations.

Sections 3–10 — Implementation Milestones Ordered

Execution Plan

- Milestones are ordered by dependency. Complete each milestone and pass its acceptance criteria before starting the next. Estimated durations assume a team of 3–5 CS students working part-time (~20 hrs/week total). Backend milestones (M1–M6) should proceed in parallel with frontend scaffolding (M7) from week 2 onward.

Milestone	Focus Area	Weeks	Dates (2026)	Status Gate
M1	Auth & Registration	1–2	Apr 7 – Apr 20	Owner can activate pharmacy account
M2	Dead Stock Listing	2–3	Apr 14 – Apr 27	Items enter matching pool with FLAGGED blocked
M3	Matching Engine	3–5	Apr 21 – May 11	2-party and 3-party cycles detected correctly
M4	Swap Ticket & Workflow	5–6	May 4 – May 18	Accept/Decline changes cycle status correctly
M5	Swap Records & Audit	6–7	May 11 – May 25	PDF generated in <60s, audit log immutable
M6	Dashboard & Alerts	7–8	May 18 – Jun 1	KPI cards accurate, near-expiry panel live
M7	React Frontend SPA	2–9	Apr 14 – Jun 8	All screens functional, Arabic RTL working
M8	Integration, I18n, NFRs	8–10	Jun 1 – Jun 20	All NFRs pass, load test passes, demo ready

Section 3 — Milestone 1: Authentication & Registration

MILESTONE 1	Authentication & Registration	Apr 7 – Apr 20
Deliverables: • POST /api/v1/auth/register — Pharmacy registration endpoint • POST /api/v1/auth/login — JWT token issuance • POST /api/v1/auth/activate — Owner activation of PENDING account • RBAC enforcement on all endpoints (Owner / Pharmacist / Employee) • JWT middleware with 8-hour expiry • Flyway V1 migration executed successfully • Unit tests: registration, login, role enforcement (TC-REG-01, TC-AUTH-01, TC-AUTH-02)		AI Agent Directive: Implement SRS-AUTH-01, SRS-AUTH-02, SRS-AUTH-03. Generate: Spring Security config with JWT filter, PharmacyController, PharmacyUserController, AuthService, JwtUtil. Enforce: PENDING accounts return HTTP 403 on any swap action. OWNER role required to activate. All endpoints secured. Password: BCrypt hash, never stored plain. Return: JWT access token + role on login.

Detailed Requirements — M1:

	Acceptance Criterion
district, owner name, commercial reg number, pharmacist contact. Status = PENDING on submit.	Account has PENDING status; owner receives activation prompt; no
ss), Pharmacist (list+swaps), Employee (data entry only). JWT 8-hour expiry.	Employee cannot accept/decline swap (HTTP 403). Owner can view

	Acceptance Criterion
unts from all swap-related actions.	Attempt by PENDING account returns HTTP 403 with activation mes

API Endpoints — M1:

Endpoint	Auth Required	Body / Response
/api/v1/auth/register	None	Body: {pharmacyName, district, ownerName, commercialRegNo, pharmacistContact, password} → 201 + phar
/api/v1/auth/login	None	Body: {email, password} → 200 + {token, role, pharmacyId, status}
/api/v1/auth/activate	Owner JWT	Body: {pharmacyId} → 200 (status becomes ACTIVE)
/api/v1/auth/me	Any JWT	→ {userId, role, pharmacyId, pharmacyStatus}
/api/v1/users	Owner JWT	Create sub-users (Pharmacist/Employee) for the same pharmacy
/api/v1/users/{id}/role	Owner JWT	Change role of a user within same pharmacy

Section 4 — Milestone 2: Dead Stock Listing

MILESTONE 2

Dead Stock Listing Subsystem

Apr 14 – Apr 27

Deliverables:

- POST /api/v1/inventory — Create dead stock listing
- GET /api/v1/inventory — List pharmacy's own items
- PATCH /api/v1/inventory/{id} — Update quantity/notes
- DELETE /api/v1/inventory/{id} — Remove listing (only if ACTIVE)
- Controlled substance detection: status → FLAGGED, admin notified
- Cold-chain exclusion: COLD_CHAIN items excluded from matching pool
- Daily scheduled job: calculate days-to-expiry, assign colour badges
- Unit tests: TC-LIST-01, TC-CTRL-01, TC-ALERT-01

AI Agent Directive:

Implement SRS-LIST-01, SRS-LIST-02, SRS-LIST-03. InventoryItem entity with mandatory fields validation (@NotNull, @Future for expiry_date). On medicine name submit: query MedicationReference table – if is_controlled=TRUE, set lock_status=FLAGGED and do NOT add to matching pool. If storage_condition=COLD_CHAIN, flag item and exclude from matching. Scheduled @Scheduled(cron='0 0 2 * * *') job updates days-to-expiry badges. Admin notification: create SystemAlert record + WebSocket push to admin session.

Mandatory Listing Fields (All Required):

Field	Type	Validation Rule	Notes
medicine_name	String	Must match MedicationReference.trade_name or scientific_name	If no match found: allow with WARNING flag
batch_number	String (alphanumeric)	Min 3 chars, max 50 chars	Manufacturer's batch ID
expiry_date	Date	Must be a FUTURE date (after today)	Reject past dates with inline error
quantity_available	Integer	Positive integer, > 0	Number of units (boxes/strips/vials)
unit	Enum	BOX STRIP VIAL PIECE	Select field in UI
storage_condition	Enum	ROOM_TEMP COLD_CHAIN	COLD_CHAIN triggers automatic exclusion
unit_price (optional)	Decimal (EGP)	Positive, 2 decimal places	Used for 'value recovered' KPI calculation
notes (optional)	String	Max 500 chars	Condition notes, packaging status

Near-Expiry Colour Badge Logic:

Condition	Badge Colour	HEX	Dashboard Placement
days_to_expiry < 30	RED	#DC2626	Top of Near-Expiry panel — urgent action required
30 ≤ days_to_expiry < 60	AMBER	#D97706	Near-Expiry panel — action recommended
60 ≤ days_to_expiry < 90	YELLOW	#CA8A04	Near-Expiry panel — monitor
days_to_expiry ≥ 90	None	—	Standard inventory list, no badge

Section 5 — Milestone 3: Matching Engine

MILESTON E 3

Geographic Matching Engine

Apr 21 – May 11

Deliverables:

- MatchingEngineService — standalone Spring @Service
- Graph construction: nodes=pharmacies, edges=supply/demand overlap
- 2-party cycle detection: $A \leftrightarrow B$ (Pharmacy A has what B needs, B has what A needs)
- 3-party cycle detection: $A \rightarrow B \rightarrow C \rightarrow A$ (Johnson's algorithm or DFS with backtracking)
- Geographic filter: Haversine distance $\leq 10\text{km}$ default (configurable 5–25km)
- Atomic inventory lock via @Transactional database lock on match found
- Scheduled job: runs every 30 minutes + triggered on every new listing
- Idempotency: running matching twice must NOT produce duplicate SwapCycles
- Unit tests: TC-MATCH-01, TC-LOCK-01, TC-LOCK-02

AI Agent Directive:

Implement SRS-MATCH-01, SRS-MATCH-02, SRS-MATCH-03. The matching engine is a SEPARATE @Service component. It must be independently scalable (separate thread pool). Algorithm: Build a directed graph where edge $A \rightarrow B$ exists if Pharmacy A has ACTIVE stock of medicine X AND Pharmacy B has medicine X on its WishListItem OR previously had it in dead stock. Detect cycles using DFS. For 3-party: use Johnson's algorithm. On cycle found: acquire pessimistic locks on all InventoryItems in cycle atomically within one @Transactional method. If any lock fails: retry once, then skip and re-queue. NEVER create duplicate SwapCycles for same items.

Matching Algorithm Specification:

The matching engine operates on a directed graph where each node is an ACTIVE pharmacy and each directed edge $A \rightarrow B$ represents 'Pharmacy A has medicine X that Pharmacy B wants (wishlist or known need).' A valid swap cycle requires ALL of the following conditions:

- ✓ All involved InventoryItems have lock_status = ACTIVE
- ✓ All involved pharmacies have account status = ACTIVE
- ✓ All pharmacy pairs are within the configurable geographic radius (Haversine formula)
- ✓ Medicine names match exactly (same GTIN or same trade_name in MedicationReference)
- ✓ No involved item has is_controlled = TRUE or storage_condition = COLD_CHAIN
- ✓ The cycle is closed — every participant both gives AND receives

Geographic Distance Calculation:

Use the Haversine formula to calculate great-circle distance between pharmacy coordinates (latitude, longitude stored in Pharmacy entity). Default radius: 10 km. Configurable per pharmacy by Owner role (5–25 km range). OpenStreetMap/Nominatim API used for geocoding pharmacy addresses to lat/long at registration time. Cache geocoded coordinates — do not re-query on every matching run.

Atomic Lock Implementation:

```
@Transactional(isolation = SERIALIZABLE)
public SwapCycle lockAndCreateCycle(List inventoryIds) {
    // 1. SELECT ... FOR UPDATE on all items (ordered by ID to prevent deadlock)
    List items = repo.findAllByIdInOrderById(inventoryIds, LockModeType.PESSIMISTIC_WRITE);
    // 2. Verify all still ACTIVE (re-check after lock)
    if (items.stream().anyMatch(i -> i.getLockStatus() != ACTIVE)) return null;
    // 3. Set all to LOCKED
```

```
items.forEach(i -> i.setLockStatus(LOCKED));  
// 4. Create SwapCycle with PENDING status and 72h expiry  
SwapCycle cycle = new SwapCycle();  
cycle.setExpiresAt(Instant.now().plus(72, HOURS));  
// 5. Save atomically  
return cycleRepo.save(cycle);  
}
```

Section 6 — Milestone 4: Swap Ticket & Confirmation Workflow

MILESTON E 4		Swap Ticket & Confirmation Workflow	May 4 – May 18
Deliverables: • WebSocket push: Swap Ticket delivered to all parties on cycle detection • GET /api/v1/swaps/{cycleId}/ticket — Retrieve Swap Ticket details • POST /api/v1/swaps/{cycleId}/accept — Accept swap (Pharmacist/Owner only) • POST /api/v1/swaps/{cycleId}/decline — Decline swap → all items revert to ACTIVE • 72-hour timer: @Scheduled job polls every 15 minutes, auto-cancels expired cycles • 24-hour reminder: WebSocket + email notification to non-responding parties • Transfer instructions generated on CONFIRMED status • POST /api/v1/swaps/{cycleId}/confirm-receipt — Both parties confirm physical receipt • Unit tests: TC-TICKET-01, TC-CONF-01, TC-TIMER-01, TC-TRANSFER-01		AI Agent Directive: Implement SRS-SWAP-01 through SRS-SWAP-04. SwapTicket is NOT a database entity – it is a DTO assembled from SwapCycle + SwapLeg + InventoryItem + Pharmacy data. Deliver via WebSocket to /user/queue/notifications (Spring STOMP). Accept/Decline: ONLY Owner or Pharmacist roles may act. Employee returns HTTP 403. All-party confirmation required: cycle status moves PENDING → PARTIAL_ACCEPT → CONFIRMED. ANY single decline: immediately set cycle to CANCELLED and atomically release all LOCKED items back to ACTIVE. 72h timer: @Scheduled every 15 min, auto-cancel cycles where expires_at < NOW(). On CONFIRMED: generate transfer instructions DTO for each party (what to send, to whom, counterparty details).	

Swap Ticket — Required Fields (ALL mandatory):

Field	Source	Display Format
Medicine Name	InventoryItem → MedicationReference.trade_name	Bold, large font — primary identifier
Batch Number	InventoryItem.batch_number	Monospace font
Expiry Date	InventoryItem.expiry_date	DD/MM/YYYY with colour badge (RED/AMBER/YELLOW)
Days Until Expiry	Calculated: expiry_date - TODAY	Number + colour indicator
Quantity Offered	SwapLeg.quantity_transferred	Number + unit (BOX/STRIP/VIAL/PIECE)
Storage Condition	InventoryItem.storage_condition	ROOM_TEMP or COLD_CHAIN badge
Offering Pharmacy Name	Pharmacy.commercial_name	Name + verified badge if license confirmed
Offering Pharmacy District	Pharmacy → Region.name	Area name
Distance	Haversine(pharmacyA.coords, pharmacyB.coords)	X.X km — displayed prominently
What You Will Receive	The other leg(s) of the SwapCycle	Separate card per incoming item
Ticket Expiry Countdown	SwapCycle.expires_at - NOW()	Live countdown: HH:MM:SS
Accept / Decline Buttons	UI action — Owner/Pharmacist only	Green ACCEPT, Red DECLINE — prominent CTAs

Swap Cycle State Machine (Full):

From State	Event	To State	Side Effects
(new)	Match found + all items locked	PENDING	Swap Tickets delivered via WebSocket to all parties
PENDING	Some but not all parties accept	PARTIAL_ACCEPT	Reminder sent to non-responding parties
PARTIAL_ACCEPT	All remaining parties accept	CONFIRMED	Transfer instructions generated and sent
PENDING / PARTIAL_ACCEPT	Any party declines	CANCELLED	All LOCKED items → ACTIVE; all parties notified
PENDING / PARTIAL_ACCEPT	Timer expires	TIMED_OUT	All LOCKED items → ACTIVE; all parties notified
CONFIRMED	Parties arrange physical transfer	IN_TRANSFER	No system action — parties self-coordinate
IN_TRANSFER	Both parties confirm receipt	COMPLETED	PDF Swap Record generated; items → COMPLETED

Section 7 — Milestone 5: Swap Records & Audit Log

MILESTONE 5

Swap Records & Immutable Audit Log

May 11 – May 25

Deliverables:

- Auto-generate PDF Swap Record within 60 seconds of COMPLETED status
- GET /api/v1/swaps/{cycleId}/record — Download Swap Record PDF
- GET /api/v1/swaps — Swap history with filters (status, date range)
- Audit log: every state transition logged with entity, actor, timestamp, IP
- AuditLog table: INSERT only, no UPDATE/DELETE permitted
- GET /api/v1/admin/audit — Audit log query (Owner + Admin only), filterable
- PDF filename format: DAWAA-[SwapID]-[YYYYMMDD].pdf
- Unit tests: TC-REC-01, TC-AUDIT-01

AI Agent Directive:

Implement SRS-REC-01 and SRS-REC-02. PDF generation: trigger via @EventListener(SwapCompletedEvent.class) within 60 seconds. Use chosen PDF library (iText 7 recommended) to generate structured document. PDF must contain ALL fields listed below – no exceptions. Store PDF as binary blob in DB OR as file in cloud storage with URL reference. AuditLog: implement as an @EntityListeners AuditingEntityListener that intercepts ALL @Entity saves/updates and writes immutable log entries. Revoke UPDATE and DELETE on audit_log table at the PostgreSQL role level.

PDF Swap Record — Required Content (ALL mandatory):

Section	Field	Value Source
Header	DawaalLink Logo + 'Official Swap Record'	Static
Header	Unique Swap ID	Format: DAWAA-[YYYYMMDD]-[6-digit-seq]
Header	Date and Time of Completion	SwapCycle completion timestamp, CLT timezone
Pharmacy A	Pharmacy name, district, commercial status	Pharmacy entity
Pharmacy A	Role in this swap: SENDER / RECEIVER	SwapLeg.direction
Pharmacy B	Pharmacy name, district, commercial status	Pharmacy entity
Pharmacy B	Role in this swap: SENDER / RECEIVER	SwapLeg.direction
Item Details	Medicine name (trade + scientific)	MedicationReference
Item Details	Batch number, expiry date, quantity	InventoryItem + SwapLeg
Item Details	Storage condition	InventoryItem.storage_condition
Footer	Generated automatically by DawaalLink	Static
Footer	This record does not constitute an official document	LegalDisclaimer

Section 8 — Milestone 6: Dashboard & Near-Expiry Alerts

MILESTONE 6	KPI Dashboard & Near-Expiry Alert System	May 18 – Jun 1
Deliverables: • GET /api/v1/dashboard — KPI aggregates: total listed, matches this week, completed swaps, EGP value recovered • GET /api/v1/inventory/near-expiry — Items within threshold (default 90 days), sorted by urgency • Real-time KPI updates via WebSocket on every relevant state change • Configurable expiry threshold per pharmacy (Owner role, 30–180 days range) • Near-Expiry panel: colour-coded (RED/AMBER/YELLOW) with urgency sort • Active Swap Matches panel: pending tickets awaiting response • Recent Swap History: last 10 swaps with status, direction, counterparty • Unit tests: TC-DASH-01, TC-ALERT-01		AI Agent Directive: Implement SRS-DASH-01 and SRS-LIST-03. Dashboard data is assembled by DashboardService from multiple repositories. KPI: value_recovered = SUM(quantity_transferred * unit_price) for COMPLETED swaps where pharmacy is SENDER. Real-time: push dashboard KPI updates via WebSocket /user/queue/dashboard on any swap state change. Near-expiry threshold: stored in Pharmacy.expiry_alert_days (default 90). Owner can update via PATCH /api/v1/pharmacy/settings. Dashboard must be the DEFAULT landing view after login (frontend redirect).

Dashboard KPI Definitions:

Card	Calculation	Scope	Update Trigger
Items Listed	COUNT(InventoryItem) WHERE pharmacy_id = ? AND lock_status IN (ACTIVE, LOCKED)	Current pharmacy	On listing create
Matches Found This Week	COUNT(SwapCycle) WHERE ANY leg.pharmacy_id = ? AND created_at > NOW() - 7 days	Current pharmacy	On new cycle
Swaps Completed	COUNT(SwapCycle) WHERE ANY leg.pharmacy_id = ? AND execution_status = COMPLETED	All time, current pharmacy	On cycle completion
Estimated Value Recovered (EUR)	SUM(leg.quantity_transferred * item.unit_price) WHERE sender = pharmacy AND status = COMPLETED	All time, SENDER legs only	On cycle completion

Section 9 — Milestone 7: React Frontend SPA

MILESTONE 7

React Single-Page Application

Apr 14 – Jun 8
(parallel with backend)

Deliverables:

- Project scaffold: Vite + React 18 + TypeScript + React Router v6
- Auth: login page, JWT storage in memory (not localStorage), axios interceptor
- Sidebar navigation (7 items as per SRS spec)
- Dashboard screen: 4 KPI cards, Near-Expiry panel, Active Matches, Recent History
- Dead Stock Listing form: all mandatory fields, inline validation
- Swap Ticket screen: all required fields, Accept/Decline CTAs, countdown timer
- Swap History screen: filterable list with status chips
- Pharmacy Profile & Settings screen
- Admin screen: flagged items review, audit log viewer
- i18n: English + Arabic (RTL) via react-i18next
- Mobile responsive: no broken layouts at ≥375px
- Cross-browser: Chrome, Firefox, Edge (latest 2 versions)

AI Agent Directive:

Scaffold React app with Vite. Use React Router v6 for SPA routing with protected routes (PrivateRoute component checking JWT role). State management: React Context + useReducer for auth state; React Query (TanStack Query) for server state. WebSocket: use SockJS + @stomp/stompjs for real-time notifications. Styling: Tailwind CSS with the brand colour palette (#6C3FC5 primary, #EDE7F6 accent). NEVER store JWT in localStorage – use in-memory store or httpOnly cookie. Arabic RTL: wrap app in . Use react-i18next with translation files for all UI strings.

Screen Inventory & Route Map:

Route	Screen	Roles	Key Components
/login	Login	Public	Email/password form, JWT handling
/register	Register Pharmacy	Public	Multi-step registration form
/dashboard	Dashboard (default)	Owner, Pharmacist	KPI cards, Near-Expiry panel, Active swaps
/inventory	My Dead Stock	Owner, Pharmacist, Employee	Table, Add/Edit/Delete, colour badges
/inventory/new	Add Dead Stock Item	Pharmacist, Employee	Listing form with all mandatory fields
/matches	Swap Matches	Owner, Pharmacist	Pending Swap Tickets, notification badge
/swaps/active	Active Swaps	Owner, Pharmacist	In-progress swaps, confirm receipt CTA
/swaps/history	Swap History	Owner, Pharmacist	Completed/cancelled history, PDF download
/profile	Pharmacy Profile	Owner	Pharmacy details, license info
/settings	Settings	Owner	Expiry threshold, user management, language
/admin/flags	Flagged Items (Admin)	System Admin	Flagged controlled substance listings
/admin/audit	Audit Log (Admin)	Owner, System Admin	Immutable log with filters

Brand Design Tokens:

Token	Value	Usage
Primary Purple	#6C3FC5	Buttons, headers, sidebar active state, badges

Token	Value	Usage
Light Purple	#EDE7F6	Card backgrounds, hover states, callout boxes
Mid Purple	#9C6FE0	Secondary actions, borders, subheadings
Dark	#1A1A2E	Body text, headings
Grey	#6B7280	Secondary text, placeholders
Grey Light	#F3F4F6	Page background, table alternate rows
Red Alert	#DC2626	Expiry <30 days, error states, Decline button
Amber Alert	#D97706	Expiry 30–60 days, warning states
Yellow Alert	#CA8A04	Expiry 60–90 days
Green	#059669	Success states, Accept button, COMPLETED badge
Font	Inter or Roboto	No serif fonts anywhere

Section 10 — Milestone 8: Integration, i18n & NFR Hardening

MILESTONE 8	Integration, Internationalisation & NFR Hardening	Jun 1 – Jun 20
Deliverables: • End-to-end integration testing: full swap lifecycle (register → list → match → accept → complete) • Arabic translations: all UI strings, error messages, notifications in react-i18next • RTL layout validation across all screens • TLS 1.3 configuration on all API endpoints • AES-256 encryption at rest for sensitive fields • Rate limiting: authentication endpoints max 10 attempts/min per IP • Load test: 500 concurrent pharmacy accounts, API response ≤500ms p95 • Cross-browser test matrix: Chrome, Firefox, Edge (latest 2 versions) • Mobile responsive test: 375px, 768px, 1024px screen widths • OpenAPI 3.0 (Swagger) documentation for all endpoints • Unit test coverage ≥70% on service layer classes • Final demo preparation and UAT with 3 pharmacists	AI Agent Directive: This milestone is integration and hardening – no new features. Run full E2E test suite covering all TC-* test cases from the traceability matrix. Security: configure Spring Security HTTPS-only, add @RateLimiter on /api/v1/auth/**, encrypt sensitive DB columns (password_hash already BCrypt; add field-level encryption for tax_id if needed). Generate OpenAPI spec via springdoc-openapi and verify all endpoints are documented. Load test with JMeter or k6: 500 virtual users, all core workflows, verify p95 < 500ms. Arabic i18n: create /public/locales/ar/translation.json with ALL keys from en/translation.json.	

NFR Acceptance Criteria (Must ALL Pass Before Demo):

NFR ID	Category	Requirement	Test Method
SRS-NFR-01	Performance	API response ≤500ms at p95 under normal load (non-matching endpoints)	JMeter/k6 load test
SRS-NFR-02	Performance	Matching engine: full run for 500 pharmacies in ≤30 seconds	Timed integration test
SRS-NFR-03	Performance	PDF Swap Record generated in ≤60 seconds of completion	Automated E2E test
SRS-NFR-04	Security	All data in transit: TLS 1.3 minimum	SSL Labs scan / config review
SRS-NFR-05	Security	Sensitive data at rest: AES-256 encryption	Code review + DB inspection
SRS-NFR-06	Security	JWT validation + RBAC enforced on every API endpoint	Role-permission test matrix (all 6 roles)
SRS-NFR-07	Security	Rate limiting: max 10 auth attempts/min per IP	Automated test (send 11 requests, verify 10th fails)
SRS-NFR-08	Privacy	No inventory data visible outside matched partners + admins	Role-permission test matrix
SRS-NFR-09	Availability	≥99% uptime during 08:00–22:00 CLT	Uptime monitoring (UptimeRobot)
SRS-NFR-10	Reliability	Matching engine idempotent — no duplicate SwapCycles	Run matching twice, verify 0 duplicates
SRS-NFR-12	Usability	Core workflows completable in ≤3 minutes by first-time user	Usability test with 3 pharmacists
SRS-NFR-13	Usability	Functional on Chrome, Firefox, Edge (latest 2 versions)	Cross-browser test suite
SRS-NFR-14	Usability	Mobile responsive: no broken layouts at ≥375px	Responsive test on 3 screen sizes
SRS-NFR-15	i18n	All UI text, errors, notifications in English AND Arabic (RTL)	Language-switch test on all 13 screens

NFR ID	Category	Requirement	Test Method
SRS-NFR-16	Scalability	≥500 concurrent pharmacy accounts without performance degradation	Load test
SRS-NFR-18	Maintainability	All REST endpoints documented in OpenAPI 3.0	Swagger UI review
SRS-NFR-19	Maintainability	≥70% unit test coverage on service layer	JaCoCo report
SRS-NFR-20	Maintainability	DB schema changes managed via Flyway migration scripts	Code review

Section 11 — Full API Contract Reference Always-On

Reference

AI AGENT DIRECTIVE – Section 11: This section is the canonical API reference. When generating controllers and service methods, every endpoint listed here must exist with the exact HTTP method, path, auth requirement, and response shape specified. Generate the OpenAPI 3.0 YAML spec from these definitions before writing any controller code.

Authentication Endpoints:

Path	Auth	Request Body	Response
/api/v1/auth/register	None	{pharmacyName, district, ownerName, commercialRegNo, pharmacistContact, email, password}	201: {pharmacyId}
/api/v1/auth/login	None	{email, password}	200: {token, role,}
/api/v1/auth/activate	Owner JWT	{pharmacyId}	200: {status:'ACT
/api/v1/auth/me	Any JWT	—	200: {userId, role,
/api/v1/users	Owner JWT	{fullName, email, password, role}	201: {userId}
/api/v1/users/{id}/role	Owner JWT	{role}	200: {userId, new

Inventory Endpoints:

Method	Path	Auth	Notes
POST	/api/v1/inventory	Pharmacist, Employee	Create listing. Returns 201 + {inventoryId, lockStatus}. Triggers matching engine run.
GET	/api/v1/inventory	Pharmacist, Owner	Own pharmacy's items. Query params: ?status=ACTIVE&expiring=90
GET	/api/v1/inventory/near-expiry	Pharmacist, Owner	Items within threshold, sorted RED→AMBER→YELLOW
PATCH	/api/v1/inventory/{id}	Pharmacist, Employee	Update quantity or notes. Only if lock_status = ACTIVE.
DELETE	/api/v1/inventory/{id}	Pharmacist, Owner	Remove listing. Only if lock_status = ACTIVE. Logs to AuditLog.

Swap Endpoints:

Method	Path	Auth	Notes
GET	/api/v1/swaps	Pharmacist, Owner	Swap history. Query params: ?status=COMPLETED&from=2026-04-01&to=2026-06-30
GET	/api/v1/swaps/{cycleId}	Pharmacist, Owner	Swap cycle details including all legs
GET	/api/v1/swaps/{cycleId}/ticket	Pharmacist, Owner	Full Swap Ticket DTO with all mandatory fields
POST	/api/v1/swaps/{cycleId}/accept	Pharmacist, Owner	Accept swap. Employee → 403. Updates leg_status.
POST	/api/v1/swaps/{cycleId}/decline	Pharmacist, Owner	Decline swap. Atomically cancels cycle + releases all locks.
POST	/api/v1/swaps/{cycleId}/confirm-receipt	Pharmacist, Owner	Confirm physical receipt. When all parties confirm → COMPLETED + PDF generated.
GET	/api/v1/swaps/{cycleId}/record	Pharmacist, Owner	Download PDF Swap Record. Returns file stream.

Admin & Utility Endpoints:

Method	Path	Auth	Notes
GET	/api/v1/admin/audit	Owner, Admin	Audit log. Filterable by pharmacyId, date range, action type.
GET	/api/v1/admin/flags	Admin	Flagged controlled substance listings awaiting review.
POST	/api/v1/admin/flags/{id}/resolve	Admin	Admin resolves flagged listing (approve listing or permanently reject).
GET	/api/v1/dashboard	Owner, Pharmacist	KPI aggregates for current pharmacy.
GET	/api/v1/wishlist	Pharmacist, Owner	Own pharmacy's wishlist items.
POST	/api/v1/wishlist	Pharmacist, Owner	Add medicine to wishlist (improves matching accuracy).
PATCH	/api/v1/pharmacy/settings	Owner	Update expiry threshold, geographic radius preference.

WebSocket Topics:

Topic (STOMP)	Direction	Payload	Trigger
/user/queue/notifications	Server → Client	SwapTicketNotification DTO	New swap match found — ticket delivered
/user/queue/notifications	Server → Client	SwapStatusUpdate DTO	Swap accepted/declined/cancelled/completed
/user/queue/notifications	Server → Client	TimerReminder DTO	24h remaining on swap ticket
/user/queue/dashboard	Server → Client	DashboardKPI DTO	Any KPI-affecting state change
/user/queue/alerts	Server → Client	NearExpiryAlert DTO	Daily job: items crossing threshold
/app/swaps/accept	Client → Server	{cycleId}	User clicks Accept on Swap Ticket
/app/swaps/decline	Client → Server	{cycleId}	User clicks Decline on Swap Ticket

Section 12 — Controlled Substance Exclusion List

Hard Constraint — EDA Compliance

- **HARD CONSTRAINT — NO EXCEPTIONS.** These substances are categorically excluded from DawaaLink under Egyptian Anti-Narcotics Law No. 182 of 1960 and EDA regulations. Any listing matching these trade names, scientific names, or categories must be automatically assigned lock_status = FLAGGED and must NEVER enter the matching pool. Admin notification required. Pharmacy notified of restriction.

AI AGENT DIRECTIVE – Section 12: Populate the MedicationReference table with ALL substances listed below, setting is_controlled = TRUE for each. Additionally, in MedicationReference, flag storage_condition = COLD_CHAIN for: Insulin (all formulations), any biological/biosimilar products, refrigerated vaccines. The SRS-LIST-02 validation must query this table on EVERY listing submission. Also hardcode these trade name patterns in a ControlledSubstanceValidator @Component as a fallback if the DB is unavailable.

Schedule I — Highest Risk (Death Penalty for Trafficking):

Trade Name(s)	Chemical Name	Category
MST, Sevredol	Morphine	Opioid Analgesic
Panadol with Codeine, Nicocodeine	Codeine	Opioid Antitussive
Sublimaze, Duragesic	Fentanyl	Synthetic Opioid
OxyContin, Percocet	Oxycodone	Opioid Analgesic
Dilaudid	Hydromorphone	Opioid Analgesic
Pethilorfan	Pethidine (Meperidine)	Opioid Analgesic
Tramadol SR, Contramal (HIGH DOSE FORMULATIONS)	Tramadol (high dose formulations)	Synthetic Opioid
Suboxone, Temgesic	Buprenorphine	Partial Opioid Agonist
Dolophine	Methadone	Opioid Substitute
Phenobarbital (some formulations)	Barbiturates	CNS Depressant
Not available medically	Heroin, Cannabis, Cocaine, Amphetamines, LSD, MDMA	Completely prohibited

Schedule II — High Risk (Special Prescription Required):

Trade Name(s)	Chemical Name	Category
Valium	Diazepam	Benzodiazepine
Xanax	Alprazolam	Benzodiazepine
Rivotril, Klonopin	Clonazepam	Benzodiazepine
Ativan	Lorazepam	Benzodiazepine
Restoril	Temazepam	Benzodiazepine
Mogadon	Nitrazepam	Benzodiazepine
Lexotanil	Bromazepam	Benzodiazepine

Trade Name(s)	Chemical Name	Category
Versed	Midazolam	Benzodiazepine (Anesthesia only)
Ritalin, Concerta	Methylphenidate	Stimulant (ADHD)
Ambien	Zolpidem	Z-drug
Imovane	Zopiclone	Z-drug
Ketalar	Ketamine	Dissociative Anesthetic
Nembutal	Pentobarbital	Barbiturate

Schedule III + Additional Exclusions:

Trade Name(s)	Chemical Name / Category	Exclusion Reason
Tramadol, Contramal (standard dose)	Tramadol	Schedule III controlled substance
Lyrica	Pregabalin	Schedule III — potential for abuse
Neurontin	Gabapentin	Schedule III — controlled access
Sustanon, Andriol	Testosterone / Anabolic Steroids	Schedule III — controlled medical use
Deca-Durabolin, Winstrol, Nandrolone, Anabolic Steroids	Anabolic Steroids	Schedule III
Librium	Chlordiazepoxide	Schedule III Benzodiazepine
All insulin formulations	Insulin	Cold-chain + biosimilar — explicitly excluded by survey (Q13)
All biosimilar products	Biosimilars	Explicitly excluded by Dr Iman Tawakol Sadeek (Q13)
Any refrigerated vaccine	Vaccines	Cold-chain storage — unverifiable remotely
Any medication requiring 2–8°C storage	Cold-chain drugs	Storage condition cannot be verified

Section 13 — Acceptance Criteria Master Checklist

QA Gate

AI AGENT DIRECTIVE – Section 13: Generate automated test cases for every TC-* item listed below. Use JUnit 5 + Mockito for Spring Boot service layer tests. Use @SpringBootTest for integration tests. Use Playwright or Cypress for E2E frontend tests. Every test must pass before milestone sign-off. Target: ≥70% coverage on service layer.

Description	Expected Result
Register new pharmacy → owner activates → account becomes ACTIVE	Status transitions PENDING → ACTIVE; all swap actions available after activation
Register with duplicate commercial registration number	HTTP 409 Conflict; no account created
Employee attempts to accept a swap	HTTP 403 Forbidden returned
PENDING account attempts to list dead stock	HTTP 403 with activation message
JWT expires (simulate 8h passage) → API call	HTTP 401; user must re-login
Submit listing with all mandatory fields complete	Item created with lock_status=ACTIVE; appears in matching pool
Submit listing with expiry_date in the past	HTTP 400; inline validation error on expiry_date field
Submit listing with empty batch_number field	HTTP 400; inline error on batch_number field
List a controlled substance (e.g. Valium / Diazepam)	lock_status=FLAGGED; item NOT in matching pool; admin notified; pharmacy notified of restriction
List cold-chain item (COLD_CHAIN storage)	Item excluded from matching pool automatically
Item expires in 25 days	RED badge applied; item in Near-Expiry panel top priority
Item expires in 45 days	AMBER badge applied
Item expires in 75 days	YELLOW badge applied
Pharmacy A lists Med X; Pharmacy B has Med X on wishlist within 10km	2-party SwapCycle created; both parties receive Swap Ticket via WebSocket
3-party cycle: A has what B needs, B has what C needs, C has what A needs	3-party SwapCycle created; all 3 parties notified simultaneously
Pharmacies are 15km apart (outside default 10km radius)	No match created (geographic filter enforced)
Run matching engine twice with same data	Only 1 SwapCycle created (idempotency)
Match found → items locked	All involved InventoryItems show lock_status=LOCKED; not visible in any new matching runs
Party declines → items released	All LOCKED items revert to ACTIVE atomically; new matching can pick them up
View Swap Ticket for a pending cycle	All 13 mandatory fields present in response DTO
Employee attempts to view Swap Ticket	HTTP 403 Forbidden
Both parties accept → cycle status	PENDING → PARTIAL_ACCEPT → CONFIRMED; transfer instructions generated for both parties
One party declines after other accepts	CANCELLED immediately; all items → ACTIVE; both parties notified
2h timer expires without response	Cycle → TIMED_OUT; all items → ACTIVE; parties notified
24h reminder sent to non-responding party	WebSocket notification + email sent at exactly 24h remaining
Swap reaches COMPLETED status	PDF generated within 60s; contains all mandatory fields; filename DAWAA-[ID]-[DATE].pdf
Download Swap Record 30 days after completion	PDF still accessible from Swap History

Description	Expected Result
Attempt DELETE on audit_log table	Permission denied at DB level
Filter audit log by pharmacy and date range	Returns correct filtered results
Complete a swap → check dashboard KPIs	Total Swaps Completed increments by 1; Value Recovered increases by correct EGP amount
Real-time KPI update via WebSocket	Dashboard updates within 2 seconds of state change without page refresh

Section 14 — Risk Register & Active Mitigations

Ongoing

	Score	Mitigation — Implement Now
without Drug ASB (6)	High (6)	Formal platform as formalising legal existing behaviour. Include audit trail in demo. Target informal endorsement letter from a local pharmacist.
stances inadvertently listed	High (6)	Hard-block at DB level (SRS-LIST-02). Fallback hardcoded validator. Admin review queue. Controlled substance list in Section 12 seeded to demo.
ns listed without Storage verification	High (6)	Inventory storage_condition field. COLD_CHAIN auto-excluded. Liability disclaimer in T&C and on every listing form.
items without Owner knowledge	High (6)	Owner-approval workflow (FR-07). Audit log captures all listing events with actor user ID.
provides fraudulent receipts	High (6)	Both parties confirm receipt before COMPLETED. Rating system in v2. Business registration verified at signup.
ensity — <20 pharmacies on demo	High (6)	Targeted WhatsApp pharmacy groups directly. Onboarding must begin 2+ months before launch. Demo with 20+ registered pharmacies minimum.
WhatsApp — Medium (4) complex	Medium (4)	Complex actions in <3 minutes (NFR-12). Usability test with 3 pharmacists before demo. Benchmark vs WhatsApp flow.
d but logistics Medium (4) expensive	Medium (4)	72h confirmation + 48h transfer window. Auto-cancel if receipt not confirmed. Re-list items automatically on cancellation.
reach	MEDIUM (3)	AES-256 at rest, TLS 1.3 in transit, RBAC on all endpoints, inventory never public.
y crackdown on medicine companies	MEDIUM (3)	Controlled substance exclusion hard-coded. Full audit trail. Position as formalising existing legal informal practice. Consult pharma law experts.

Final Agent Directive — Execution Order [Read Last](#)

MASTER AI AGENT EXECUTION SEQUENCE: STEP 1: Read Section 0 (architecture) and Section 12 (controlled substances) in full. STEP 2: Confirm 2 open decisions with team (PDF library + notification provider). Record in project README. STEP 3: Execute Section 2 – generate Flyway V1__init_schema.sql and all 9 JPA @Entity classes. STEP 4: Execute Milestone 1 (Section 3) – Auth, RBAC, JWT. All TC-REG and TC-AUTH tests must pass. STEP 5: Execute Milestone 2 (Section 4) – Dead Stock Listing. TC-LIST, TC-CTRL, TC-ALERT must pass. STEP 6: Execute Milestone 3 (Section 5) – Matching Engine. TC-MATCH, TC-LOCK must pass. STEP 7: Execute Milestone 4 (Section 6) – Swap Ticket & Workflow. TC-TICKET, TC-CONF, TC-TIMER must pass. STEP 8: Execute Milestone 5 (Section 7) – PDF Records & Audit. TC-REC, TC-AUDIT must pass. STEP 9: Execute Milestone 6 (Section 8) – Dashboard & Alerts. TC-DASH must pass. STEP 10: Scaffold React SPA (Milestone 7, Section 9) in parallel from Step 4 onward. STEP 11: Execute Milestone 8 (Section 10) – Integration, I18n, NFR hardening. All NFRs must pass. STEP 12: Run full E2E test suite (all TC-* from Section 13). Fix all failures. STEP 13: Generate OpenAPI 3.0 spec and verify all Section 11 endpoints are documented. STEP 14: Load test (500 users, all workflows, p95 < 500ms). STEP 15: Demo-ready. Invite 3 pharmacists for usability test. Confirm <3min per workflow. AT ALL TIMES: Controlled substances in Section 12 are NEVER listable. No cash processing. No public inventory. Greater Cairo only.